

Designing Software Architecture with UML

Week 5 — the decisions that are expensive to change,
and how to draw them so others can build from them

Mr. Solomon Hombiku

Computing & Emerging Technologies · Building PNG's Digital Talent

From requirements to a structure you can build

Where we are

Week 2: what the client needs (the SRS).

Week 3: how the work is organised over time.

Week 4: how we prove it works.

Week 5: the shape of the system itself — and how to draw it.

- 1 Explain what software architecture is and why its decisions are costly to reverse
- 2 Show how non-functional requirements drive architectural choices
- 3 Describe layered, client–server, MVC and repository patterns
- 4 Model structure with UML package, component and deployment diagrams
- 5 Design an offline-capable architecture for a PNG client
- 6 Produce and peer-review a diagram using diagrams.net or PlantUML

Maps to ULO2 and ULO4 · this week's output feeds Task 4 (15%, due Week 11)

The decisions you cannot cheaply undo

“The design process for identifying the sub-systems making up a system and the framework for sub-system control and communication.”

— Ian Sommerville, *Software Engineering* (10th ed.)

The test of an architectural decision

“If we got this wrong, how much would it cost to change in six months?”
Expensive to reverse = architectural. Cheap to reverse = ordinary design.

Architecture IS

- Which major parts exist, and what each is responsible for
- How the parts talk to each other
- Where each part runs
- The decisions that satisfy the non-functional requirements

Architecture is NOT

- The detail of every class and method (that is design, Weeks 9–10)
- A picture drawn after the code is written
- A folder structure
- Something only 'architects' at big companies do

Non-functional requirements decide the shape

“Must work with no internet for a full buying day”

Local database on the device; deferred sync; conflict rules

“No data loss on sudden power failure”

Transactional writes; commit before acknowledging; no in-memory-only state

“Must run on one existing laptop”

Single-machine deployment; no server tier; modest memory footprint

“Only the bursar may approve a refund”

An authorisation layer every write passes through — not a UI checkbox

This is why Week 2 came first. An architecture that ignores the NFRs is a drawing, not a design.

Six questions an architecture must answer

1

What are the major parts?

Sub-systems and their single responsibilities

2

How do they communicate?

Direct calls, shared database, messages, files

3

Where does data live?

One store or several; on the device or on a server

4

Where does each part run?

Laptop, phone, depot machine, town office, cloud

5

How is it kept secure?

Who may do what, and where that is enforced

6

What happens when it fails?

Power loss, no signal, disk full, duplicate sync

Answer these six for your Task 2 client and you have an architecture — Lab 4 walks you through it.

Four architectural patterns worth knowing

Layered

UI → logic → data. Each layer uses only the one below.

Good: easy to understand, testable, replaceable layers.

Watch: can become slow if every call crosses all layers.

Client-server

Many clients talk to a shared server holding the data.

Good: central data, many users.

Watch: needs a network — a hard sell at a PNG depot.

MVC

Model (data & rules), View (screen), Controller (input).

Good: separates screen from rules; suits UI-heavy apps.

Watch: over-used; adds indirection on small tools.

Repository

Parts share one central data store rather than calling each other.

Good: simple integration, one source of truth.

Watch: the store becomes the bottleneck and single point of failure.

Three UML diagrams that describe structure

1

Package diagram

How the system is divided into logical groups

“What are the parts, and what depends on what?”

2

Component diagram

The replaceable units and the interfaces between them

“What plugs into what, and through which contract?”

3

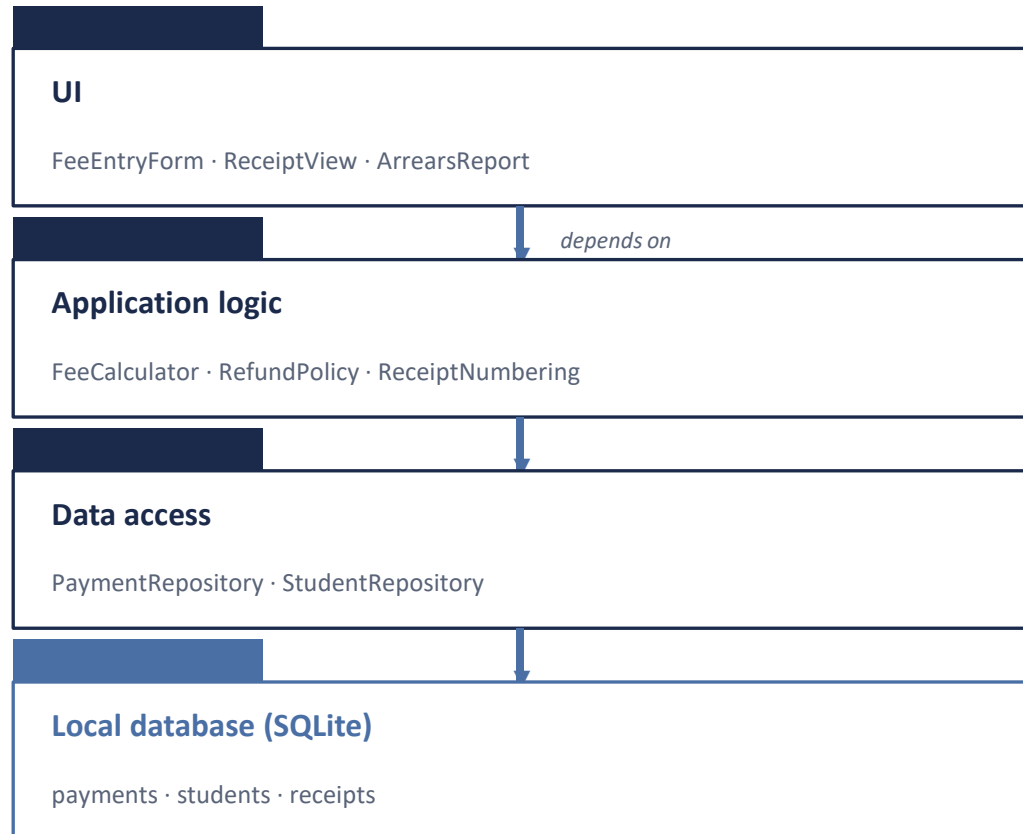
Deployment diagram

Which hardware each part runs on

“Where does this actually run — and what if the link is down?”

Weeks 9–10 add the behavioural diagrams — use case, class, state and activity — and the “4+1” view model. This week is the structural half of Task 4.

Package diagram — the layered fee system



Read it top-down

Each package uses only the one below it. UI never touches the database directly.

Why this helps

Swap SQLite for another store and only Data access changes. Test logic with no screen at all.

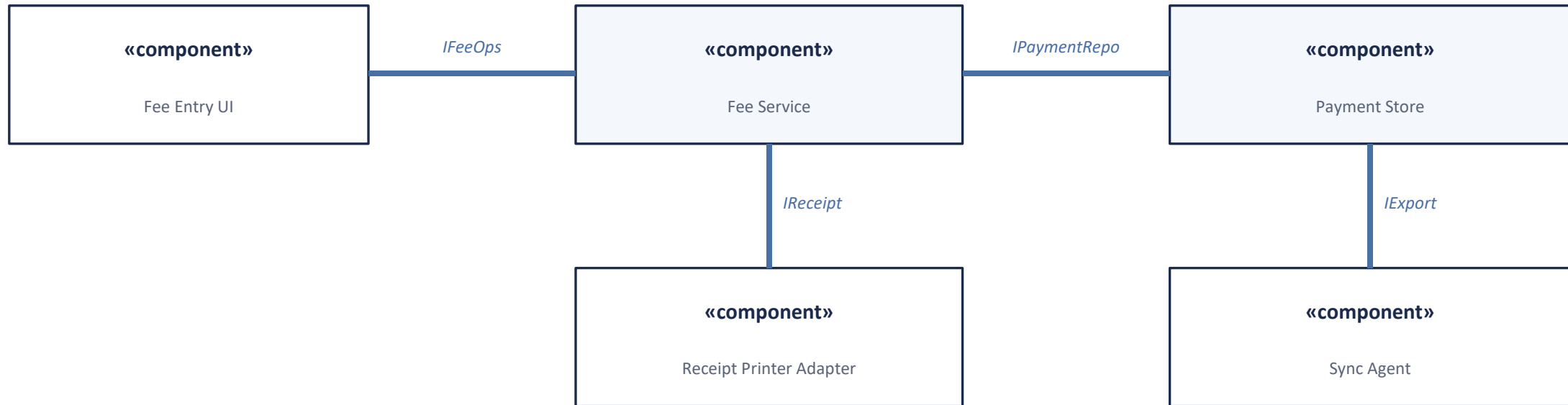
The rule to enforce

No upward arrows. If logic calls the UI, the layering is already broken.

For your Task 2 client

Three or four packages is right. Twelve means you are drawing classes, not architecture.

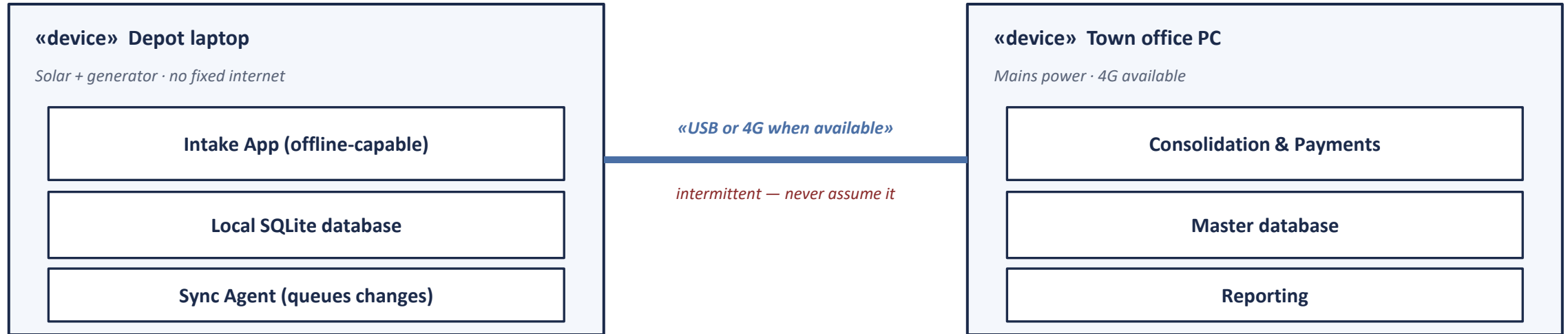
Component diagram — units and interfaces



Why interfaces, not arrows to concrete things

Fee Service depends on `IPaymentRepo` — a contract — not on `SQLite`. That is what lets you swap the store, and it is exactly what made `FeeCalculator` unit-testable in Week 4. Architecture and testability are the same decision seen twice.

Deployment diagram — where it runs, and what breaks



This is the diagram that saves PNG projects

It forces you to draw the link that is not always there — and then answer: what can staff still do while it is down? Where is the truth held? What happens when the same intake syncs twice?

A client–server design with one central database would look tidier on paper and fail on day one at the depot.

Four architectural habits that survive here

Offline-first, not offline-tolerant

Design so the local device is fully functional alone, and treat the network as an optional accelerator. Retrofitting offline onto an online design is a rewrite.

Decide where the truth lives

When depot and town disagree, which record wins? Write that rule down before you code it — it is a business decision, not a technical one.

Make sync idempotent

The same batch will be sent twice. Give every record a stable identity so a repeat import changes nothing.

Assume the power dies mid-write

Commit transactionally and acknowledge only after the write is durable. 'It worked on my laptop' is not a power-failure test.

These four turn up again in Task 3 (technology selection, Week 10) and in Task 4's architecture marks.

Two free tools — both work offline

diagrams.net (draw.io) — desktop app

- Apache 2.0 licence, free, no account needed
- Install the DESKTOP version — it runs with no internet at all
- Has UML shape libraries built in: package, component, node
- Save the .drawio file into your portfolio — it is text, so Git can track your changes
- Export PNG for your Task 4 report

PlantUML — diagrams written as text

```
@startuml
package "UI" {
    [Fee Entry Form]
}
package "Logic" {
    [Fee Service]
}
package "Data" {
    [Payment Store]
}
[Fee Entry Form] --> [Fee Service]
[Fee Service] --> [Payment Store]
@enduml
```

Text diagrams diff and merge in Git — you can see exactly what changed between versions. Ideal for the portfolio.

Either tool is fine for Lab 4 and Task 4 — pick one.

AI will draw it. It will not own the consequences.

Where it helps

- Turning your SRS into a first-draft package structure
- Generating PlantUML text you then correct
- Suggesting a pattern and naming its trade-offs
- Explaining an unfamiliar diagram someone else drew

Where it fails here

- Defaults to cloud client–server — the wrong answer for a depot
- Invents components the requirements never asked for
- Silently ignores power loss, sync duplication and who-wins rules
- Produces a tidy diagram that cannot be built on one laptop

Live demo: we ask an AI for the Waghi depot architecture — then count how many of our four PNG habits it breaks.

Design it, draw it, defend it

Lab 4: Architecture & UML (Moodle)

Answer the six design questions for YOUR Task 2 client, then draw a package and a deployment diagram. Commit to week05/.

*This week***UML peer feedback (formative, F2F)**

Swap diagrams with a partner and review against the checklist — upward arrows, missing failure paths, unlabelled links.

*In session***Homework (Moodle, formative)**

Rewrite one architectural decision as a short justification: what you chose, what you rejected, and which NFR decided it.

*Before Week 6***Keep the portfolio green**

Week 4 tests still pass; add this week's diagram files. Text diagrams (.drawio, .puml) diff nicely in Git.

This week

Next week: the estimation workshop — turning this design into a defensible schedule for Task 2 (due end of Week 8).

Where today's material comes from

- Sommerville, I. (2016). Software Engineering, 10th ed., Chapter 6 “Architectural Design”. Pearson. Free slides & videos: <https://software-engineering-book.com/slides/>
- [uml-diagrams.org](https://www.uml-diagrams.org) — UML 2.5 reference for package, component and deployment diagrams: <https://www.uml-diagrams.org>
- [diagrams.net \(draw.io\)](https://www.drawio.com) — free, Apache 2.0; install the desktop app for offline use: <https://www.drawio.com>
- PlantUML — open-source text-to-diagram tool; component and deployment syntax: <https://plantuml.com>
- Bass, L., Clements, P. & Kazman, R. (2021). Software Architecture in Practice, 4th ed. Addison-Wesley. (Library copy — the standard reference on quality attributes driving architecture.)
- Google Cloud (2025). The New Software Development Lifecycle: Vibe Coding (whitepaper) — cached on Moodle; basis of the AI-era section.

Offline access: the diagrams.net desktop installer, PlantUML jar, Chapter 6 slides and all Week 5 worksheets are cached on Moodle. Both drawing tools run with no internet.